SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – COMPARATIVE ANALYSIS (5 Questions)

Course Code:	DSA06	**Course Title:**	Data Handling and Visualization
CO Assessed:	CO4 – Naturalization (BL5)	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	**Total Marks:**	50 (5 Questions × 10 Marks)
CO4 Statement:	*Evaluate visualization designs based on perception, cognition, and effectiveness principles (BL5).*		
Student Name:	RENEE VINCENT ASR	**Reg. No.:**	192424161
Section / Batch:	BTECH AI & DS	**Date:**	14-08-2026

Code of Conduct

I Renee Vincent ASR-192424161 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Renee Vincent

Instructions

- This test contains 5 questions, each carrying 10 marks. Total: 50 Marks.
- No negative marking. Unanswered questions carry zero marks.
- No calculators or electronic devices permitted.
- Duration: 60 minutes.

Section / Topic	Focus	Q Nos	Marks
Scatterplots & Hexbin Plots	Comparative analysis of scatterplots and hexbin plots for large-scale clustered data visualization, density estimation, and overplotting handling	1	10
Dimension Reduction Techniques	PCA, t-SNE, and UMAP comparison for high-dimensional data reduction, clustering, and structure preservation	2	10
Correlograms & Network Graphs	Correlation visualization, threshold-based network construction, and multivariate relationship analysis	3	10
Time Series Analysis & Decomposition	Moving Average, Detrending, and STL decomposition for trend extraction, seasonality analysis, and forecasting suitability	4	10
Geospatial Data Visualization	Choropleth maps, cartograms, and geospatial heatmaps for spatial pattern analysis and hotspot detection	5	10
	GRAND TOTAL:		**50 Marks**

Annexure A – Comparative Analysis

1. A retail company maintains transaction records containing customer spending amounts. Construct a Histogram, Density Plot, and ECDF (Empirical Cumulative Distribution Function) and perform a comparative analysis.

Analysis Parameters:
A. Distribution shape identification
B. Detection of skewness and outliers
C. Representation of cumulative information
D. Sensitivity to sample size
E. Ease of interpretation

2. A healthcare dataset contains patient attributes such as age, blood pressure, cholesterol, and BMI. Generate a Pair Plot, Parallel Coordinates Plot, and Scatterplot Matrix and compare their effectiveness in multivariate analysis.

Analysis Parameters:
A. Relationship identification among variables
B. Detection of clusters and anomalies
C. Scalability with increasing dimensions
D. Visual complexity and readability
E. Suitability for exploratory data analysis

3. A manufacturing dataset contains measurements from multiple sensors. Create a Heatmap, Correlogram, and Clustered Heatmap to analyze relationships among variables and perform a comparative analysis.

Analysis Parameters:
A. Correlation representation
B. Pattern and anomaly detection
C. Grouping of related variables
D. Scalability to large feature sets
E. Interpretability of results

4. A financial time-series dataset contains daily stock prices recorded over several years. Apply Line Plot, Moving Average Plot, and STL Decomposition and perform a comparative analysis.

Analysis Parameters:
A. Trend identification
B. Seasonality detection
C. Noise reduction capability
D. Responsiveness to changes
E. Suitability for forecasting

5. A social network dataset contains interactions among users. Construct a Network Graph, Force-Directed Graph, and Adjacency Matrix Visualization and compare their effectiveness.

Analysis Parameters:
A. Relationship representation
B. Community detection capability
C. Scalability with network size
D. Visual clutter management
E. Identification of influential nodes

1. Customer Spending Analysis

Comparative Analysis

Analysis Parameter	Histogram	Density Plot	ECDF
A. Distribution shape	Shows shape using bins	Shows smooth distribution curve	Shows cumulative distribution
B. Skewness & outliers	Easily identifies skewness and extreme values	Shows skewness smoothly but outliers may be less obvious	Extreme values appear near the ends
C. Cumulative information	Does not directly show cumulative values	Does not show cumulative proportion	Directly shows proportion below each spending value
D. Sample size sensitivity	Sensitive to bin width, especially with small samples	Sensitive to sample size and smoothing	Relatively less sensitive because it uses every observation
E. Ease of interpretation	Very easy for beginners	Easy for understanding overall shape	Very useful for percentile and cumulative comparisons

A. Distribution Shape Identification

Histogram:

Divides spending amounts into intervals and shows how many customers fall within each interval.

Density Plot:

Provides a smooth curve representing the estimated probability distribution. Peaks indicate areas where spending values are concentrated.

ECDF:

Shows the cumulative proportion of customers whose spending is less than or equal to a particular value.

B. Skewness and Outliers

The histogram is particularly useful for identifying right-skewed or left-skewed spending patterns. For example, a long right tail indicates that a small number of customers spend substantially more

than most customers.

The density plot makes the overall skewness easier to see, but individual outliers can become less visually prominent because of smoothing.

The ECDF preserves every observation and can make unusually high or low spending values visible at the extreme ends.

C. Cumulative Information

The ECDF is the best visualization for cumulative information.

For example, if the ECDF reaches 0.80 at $500, it means approximately 80% of customers spent $500 or less.

D. Sensitivity to Sample Size

A histogram can change substantially depending on the selected number of bins, particularly with a small dataset.

A density plot can also become unstable with small samples because its smooth curve depends on bandwidth selection.

The ECDF does not require bins or smoothing and therefore provides a direct representation of the observed sample.

E. Ease of Interpretation

- Histogram: Best for beginners and quick distribution analysis.
- Density Plot: Best for comparing the overall shapes of distributions.
- ECDF: Best for cumulative percentages, percentiles, and threshold-based analysis.

Overall Conclusion

For customer spending data, no single plot is sufficient for every purpose. The histogram gives the clearest view of frequency and potential skewness, the density plot provides a smoother representation of the distribution, and the ECDF is the strongest choice for understanding cumulative proportions and spending thresholds. Using all three together gives a more complete understanding of customer spending behavior.

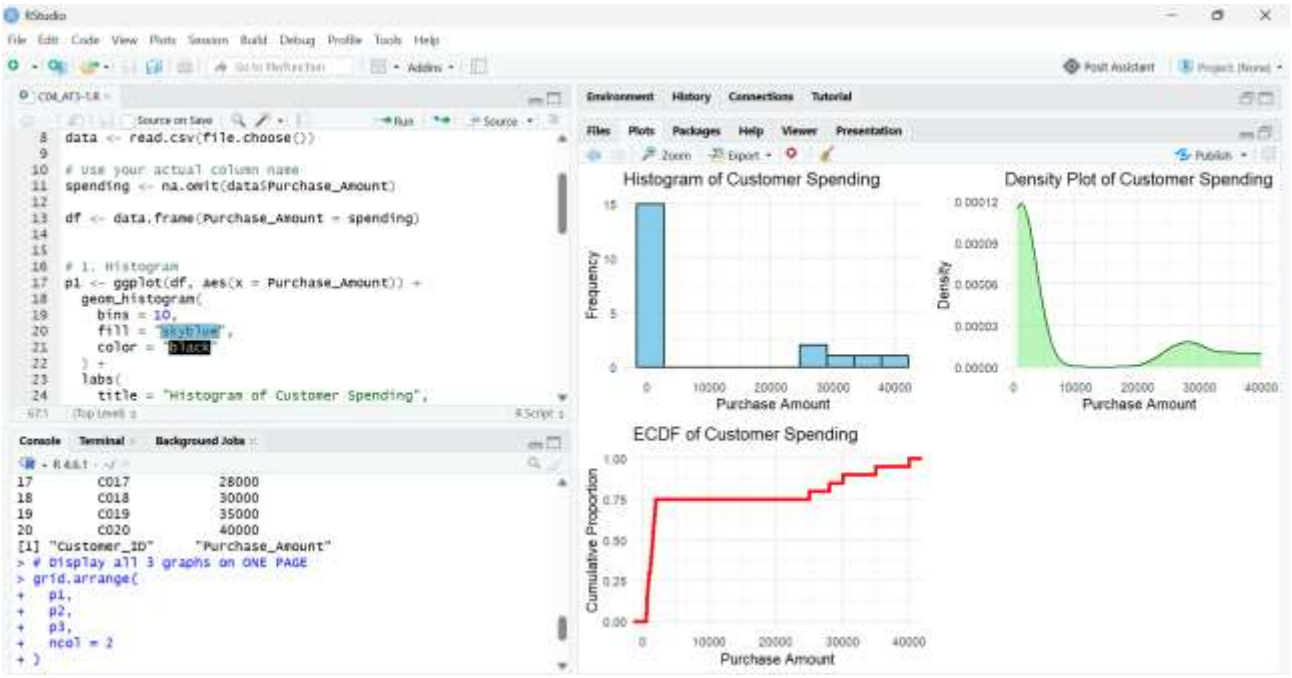
Code

```
# Customer Spending Analysis
# Histogram + Density Plot + ECDF on ONE PAGE

library(ggplot2)
```

```
library(gridExtra)

# Import CSV
data <- read.csv(file.choose())

# Use your actual column name
spending <- na.omit(data$Purchase_Amount)

df <- data.frame(Purchase_Amount = spending)


# 1. Histogram
p1 <- ggplot(df, aes(x = Purchase_Amount)) +
  geom_histogram(
    bins = 10,
    fill = "skyblue",
    color = "black"
  ) +
  labs(
    title = "Histogram of Customer Spending",
    x = "Purchase Amount",
    y = "Frequency"
  ) +
  theme_minimal()


# 2. Density Plot
p2 <- ggplot(df, aes(x = Purchase_Amount)) +
  geom_density(
    fill = "lightgreen",
    color = "darkgreen",
    alpha = 0.6
  ) +
  labs(
    title = "Density Plot of Customer Spending",
    x = "Purchase Amount",
    y = "Density"
  ) +
  theme_minimal()


# 3. ECDF
p3 <- ggplot(df, aes(x = Purchase_Amount)) +
  stat_ecdf(
    color = "red",
    linewidth = 1.2
```

```
  ) +
  labs(
    title = "ECDF of Customer Spending",
    x = "Purchase Amount",
    y = "Cumulative Proportion"
  ) +
  theme_minimal()


# Display all 3 graphs on ONE PAGE
grid.arrange(
  p1,
  p2,
  p3,
  ncol = 2
)
```

Output



2. Healthcare Multivariate Analysis

Comparative Analysis

Parameter	Pair Plot	Parallel Coordinates	Scatterplot Matrix
A. Relationship identification	Excellent for examining pairwise	Shows relationships across all variables simultaneously	Excellent for pairwise relationships

	relationships		
B. Clusters & anomalies	Can reveal clusters and outliers	Good for detecting unusual patient profiles	Good for identifying outliers
C. Scalability	Becomes crowded as variables increase	Handles many dimensions relatively well	Becomes difficult to read with many variables
D. Visual complexity	Moderate	High, especially with many observations	Moderate to high
E. Exploratory analysis	Excellent	Very useful for multidimensional patterns	Excellent for pairwise exploration

A. Relationship Identification

The pair plot and scatterplot matrix are particularly effective for identifying relationships between variables.

For example, you can examine whether:

- Age is related to blood pressure.
- BMI is related to cholesterol.
- Age is associated with cholesterol.
- Blood pressure increases with BMI.

Each pair of variables is displayed as a separate scatter plot.
The parallel coordinates plot instead represents each patient as a line passing through all variables, allowing several variables to be examined simultaneously.

B. Clusters and Anomalies

The pair plot can reveal clusters where patients have similar combinations of age, blood pressure, cholesterol, and BMI.
The parallel coordinates plot is especially useful for identifying unusual patient profiles. A line that behaves very differently from the majority may represent an anomalous observation.
The scatterplot matrix can also reveal individual observations that are far away from the main group.

C. Scalability

When the number of variables increases, pair plots and scatterplot matrices become increasingly large because they display many variable combinations.
For example, with four variables, there are only a few pairwise relationships to examine. With dozens of variables, the matrix can become difficult to interpret.
A parallel coordinates plot scales better to higher-dimensional data, because all variables can be

represented along parallel axes.

D. Visual Complexity and Readability

Pair Plot:

Easy to understand because each cell represents a relationship between two variables. However, it can become crowded.

Parallel Coordinates:

Can display many dimensions in one figure, but overlapping lines can make the visualization difficult to read.

Scatterplot Matrix:

Very intuitive for pairwise relationships, but the large number of plots can create visual complexity.

E. Suitability for Exploratory Data Analysis

Pair Plot — Best overall

A pair plot is highly suitable for initial healthcare data exploration because it allows researchers to quickly examine:

- Correlations
- Distributions
- Relationships
- Potential outliers

Parallel Coordinates — Best for high dimensions
Useful when you want to examine complete patient profiles across many variables.

Scatterplot Matrix — Best for pairwise relationships
Useful when the primary objective is to compare relationships between individual numerical variables.

Overall Conclusion

For this healthcare dataset, the Pair Plot is the most effective overall exploratory visualization because it provides distributions and pairwise relationships in a single view. The Scatterplot Matrix is similarly effective for identifying correlations and outliers, while the Parallel Coordinates Plot becomes more valuable when the dataset contains many dimensions and complete patient profiles need to be compared.

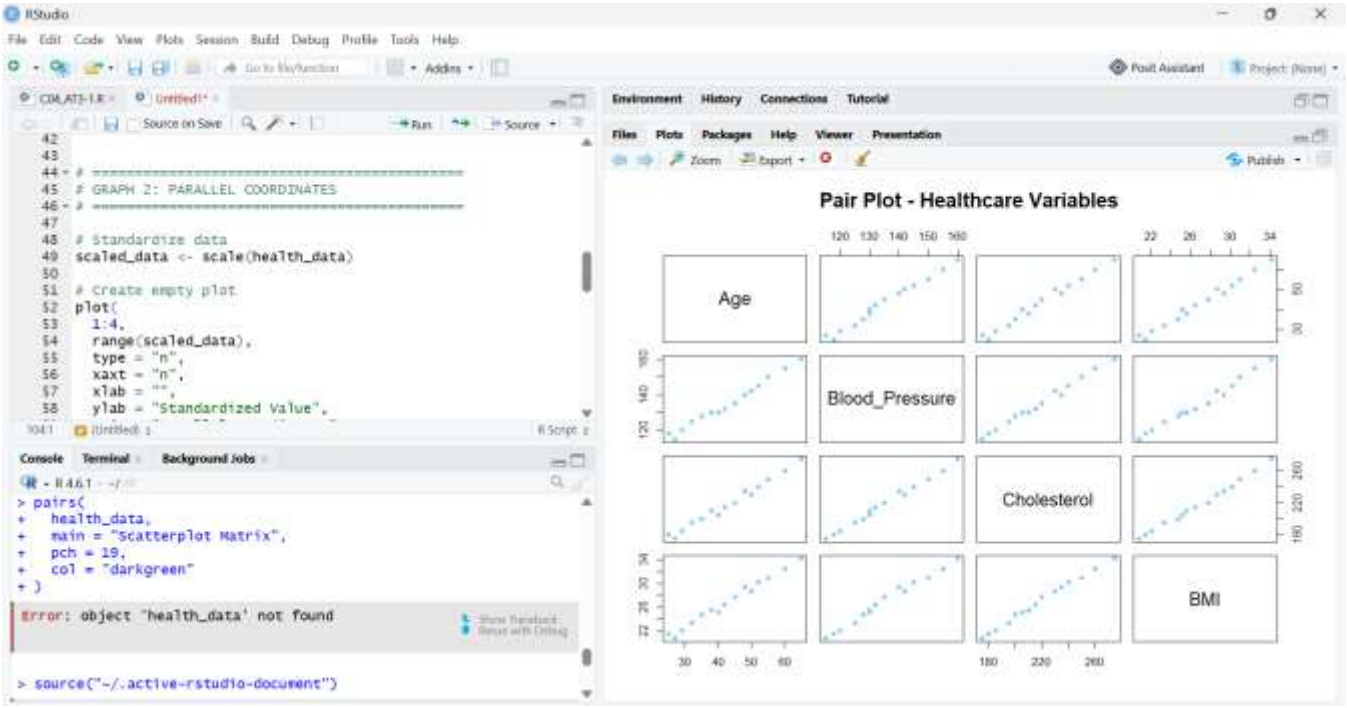
Code

```
# ============================================
# HEALTHCARE MULTIVARIATE ANALYSIS
# ALL 3 GRAPHS ON ONE PAGE
# ============================================

# Import dataset
data <- read.csv(file.choose())

# Select healthcare variables
health_data <- data[, c(
  "Age",
  "Blood_Pressure",
  "Cholesterol",
  "BMI"
)]

# Remove missing values
health_data <- na.omit(health_data)


# ============================================
# CREATE 3 PLOTS ON ONE PAGE
# ============================================

# 1 row and 3 columns
par(
  mfrow = c(1, 3),
  mar = c(4, 4, 3, 1)
)


# ============================================
# GRAPH 1: PAIR PLOT
# ============================================

pairs(
  health_data,
  main = "Pair Plot",
  pch = 19,
  col = "skyblue"
)


# ============================================
# GRAPH 2: PARALLEL COORDINATES
```

```
# ============================================

# Standardize data
scaled_data <- scale(health_data)

# Create empty plot
plot(
  1:4,
  range(scaled_data),
  type = "n",
  xaxt = "n",
  xlab = "",
  ylab = "Standardized Value",
  main = "Parallel Coordinates"
)

# X-axis labels
axis(
  1,
  at = 1:4,
  labels = c(
    "Age",
    "Blood Pressure",
    "Cholesterol",
    "BMI"
  ),
  cex.axis = 0.7
)

# Draw patient lines
for (i in 1:nrow(scaled_data)) {
  lines(
    1:4,
    scaled_data[i, ],
    type = "o",
    pch = 16,
    col = "steelblue"
  )
}


# ============================================
# GRAPH 3: SCATTERPLOT MATRIX
# ============================================

pairs(
```

```
  health_data,
  main = "Scatterplot Matrix",
  pch = 19,
  col = "darkgreen"
)


# ============================================
# RESET PLOT SETTINGS
# ============================================

par(mfrow = c(1, 1))
```

Output



3. Manufacturing Sensor Analysis

Comparative Analysis

Parameter	Heatmap	Correlogram	Clustered Heatmap
A. Correlation representation	Indirect	Excellent	Indirect
B. Pattern & anomaly detection	Excellent	Good	Excellent
C. Grouping related variables	Limited	Shows relationships	Excellent

D. Scalability	Good	Good	Very good
E. Interpretability	Easy	Very easy for correlations	Moderate

A. Correlation Representation

The correlogram is the best visualization for understanding correlations. It directly displays the correlation coefficient between every pair of sensor variables.

For example, if Temperature and Pressure have a value close to +1, they have a strong positive relationship.

B. Pattern and Anomaly Detection

The heatmap makes unusual sensor readings easy to identify because they appear as different color intensities.

The clustered heatmap is even more useful because similar sensors are placed close together, making groups and unusual observations easier to identify.

C. Grouping Related Variables

The clustered heatmap is the strongest choice for grouping related sensors. Hierarchical clustering organizes similar observations into groups.

D. Scalability

For a small number of sensors, all three visualizations are easy to interpret. As the number of sensors and features increases, a simple heatmap or correlogram can become crowded. A clustered heatmap is generally more useful for finding groups in larger datasets.

E. Interpretability

- Heatmap: Easy to understand overall sensor patterns.
- Correlogram: Best for interpreting correlations.
- Clustered Heatmap: Best for identifying groups and similar sensor behavior.

Overall Conclusion

The correlogram is most effective when the primary goal is to understand relationships between sensor variables. The heatmap is useful for detecting patterns and unusual sensor readings, while the clustered heatmap provides the most comprehensive view for identifying groups of similar sensors and variables. Therefore, using all three together provides a stronger multivariate analysis than relying on any single visualization.

Code

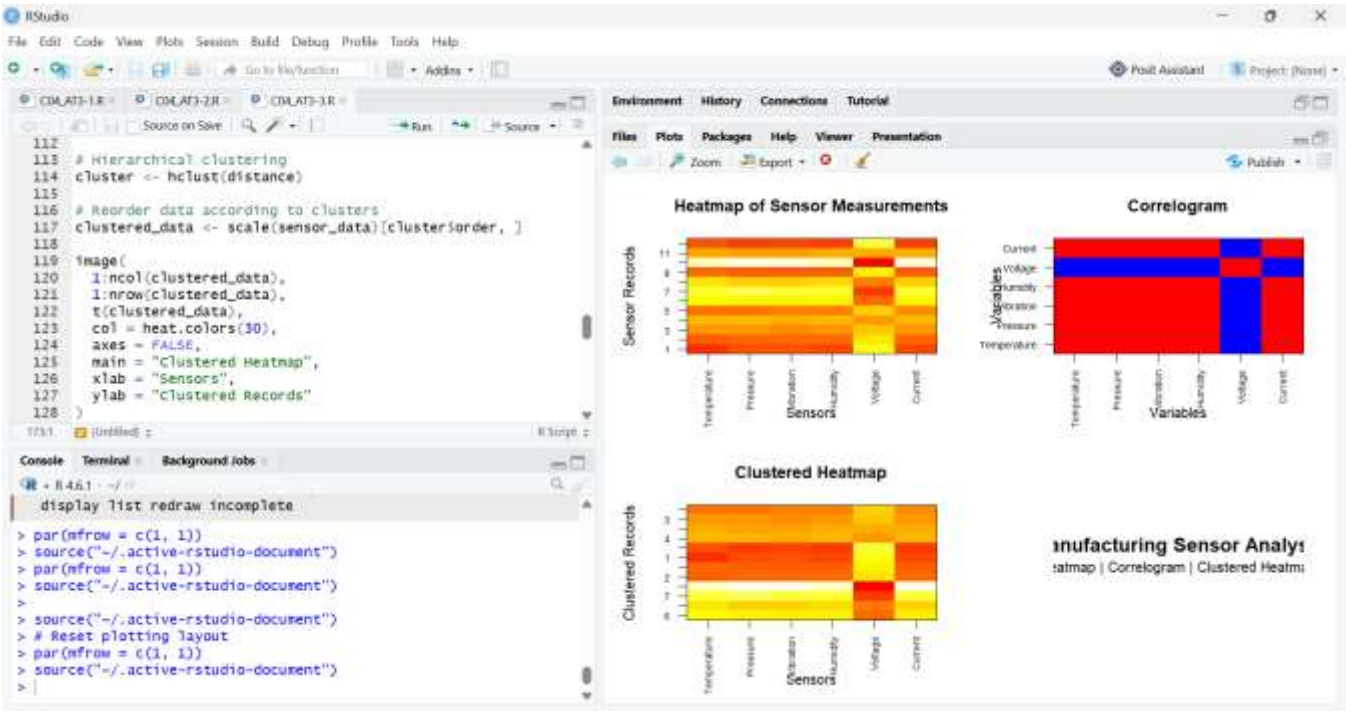
```
# ============================================
# MANUFACTURING SENSOR ANALYSIS
# 3 GRAPHS ON ONE PAGE
# ============================================

# Import dataset
data <- read.csv(file.choose())

# Select numerical variables
sensor_data <- data[, c(
  "Temperature",
  "Pressure",
  "Vibration",
  "Humidity",
  "Voltage",
  "Current"
)]

# Remove missing values
sensor_data <- na.omit(sensor_data)

# ============================================
# CREATE 2 x 2 PAGE
# ============================================

par(
  mfrow = c(2, 2),
  mar = c(5, 5, 4, 2)
)


# ============================================
# 1. HEATMAP
# ============================================

image_data <- scale(sensor_data)

image(
  1:ncol(image_data),
  1:nrow(image_data),
  t(image_data),
  col = heat.colors(30),
  axes = FALSE,
  main = "Heatmap of Sensor Measurements",
  xlab = "Sensors",
```

```
  ylab = "Sensor Records"
)

axis(
  1,
  at = 1:ncol(sensor_data),
  labels = colnames(sensor_data),
  las = 2,
  cex.axis = 0.7
)

axis(
  2,
  at = 1:nrow(sensor_data),
  labels = rownames(sensor_data),
  las = 1,
  cex.axis = 0.7
)

box()


# ==========================================
# 2. CORRELOGRAM
# ==========================================

cor_matrix <- cor(sensor_data)

image(
  1:ncol(cor_matrix),
  1:ncol(cor_matrix),
  t(cor_matrix),
  col = colorRampPalette(
    c("blue", "white", "red")
  )(100),
  axes = FALSE,
  main = "Correlogram",
  xlab = "Variables",
  ylab = "Variables"
)

axis(
  1,
  at = 1:ncol(cor_matrix),
  labels = colnames(cor_matrix),
  las = 2,
```

```
  cex.axis = 0.7
)

axis(
  2,
  at = 1:ncol(cor_matrix),
  labels = colnames(cor_matrix),
  las = 2,
  cex.axis = 0.7
)

box()


# ============================================
# 3. CLUSTERED HEATMAP
# ============================================

# Calculate distance between sensor records
distance <- dist(scale(sensor_data))

# Hierarchical clustering
cluster <- hclust(distance)

# Reorder data according to clusters
clustered_data <- scale(sensor_data)[cluster$order, ]

image(
  1:ncol(clustered_data),
  1:nrow(clustered_data),
  t(clustered_data),
  col = heat.colors(30),
  axes = FALSE,
  main = "Clustered Heatmap",
  xlab = "Sensors",
  ylab = "Clustered Records"
)

axis(
  1,
  at = 1:ncol(clustered_data),
  labels = colnames(sensor_data),
  las = 2,
  cex.axis = 0.7
)
```

```
axis(
  2,
  at = 1:nrow(clustered_data),
  labels = cluster$order,
  las = 1,
  cex.axis = 0.7
)

box()


# ============================================
# 4. EMPTY FOURTH PANEL
# ============================================

plot.new()

text(
  0.5,
  0.65,
  "Manufacturing Sensor Analysis",
  cex = 1.4,
  font = 2
)

text(
  0.5,
  0.45,
  "Heatmap | Correlogram | Clustered Heatmap",
  cex = 1
)


# Reset plotting layout
par(mfrow = c(1, 1))
```

Output



4. Financial Time-Series Analysis

Comparative Analysis

Parameter	Line Plot	Moving Average	STL Decomposition
A. Trend identification	Good	Excellent	Excellent
B. Seasonality detection	Limited	Limited	Excellent
C. Noise reduction	Poor	Good	Excellent
D. Responsiveness to changes	Excellent	Good	Moderate
E. Forecasting suitability	Basic	Good	Very good

A. Trend Identification

The line plot provides the original stock-price movement and makes overall increases or decreases easy to observe.

The moving average smooths short-term fluctuations and makes the underlying trend clearer.
STL decomposition separates the trend component from seasonal and irregular components, making it the most informative for trend analysis.

B. Seasonality Detection

The line plot can show repeating patterns, but they may be difficult to distinguish from random

fluctuations.

The moving average reduces short-term variation but can also hide seasonal patterns.
STL decomposition is the best method for detecting seasonality because it explicitly separates the seasonal component.

C. Noise Reduction

The original line plot contains all short-term fluctuations and noise.

The moving average reduces noise by averaging neighboring observations.
STL also separates the irregular component from the trend and seasonal components, providing a detailed view of the noise.

D. Responsiveness to Changes

The line plot responds immediately to every price change.
A moving average responds more slowly because it uses multiple observations to calculate each value.

STL provides a smoothed trend and therefore may not reflect sudden changes as quickly as the original line plot.

E. Suitability for Forecasting

The line plot is useful for understanding historical movements but is not sufficient by itself for forecasting.

Moving averages can provide a simple baseline for forecasting.
STL decomposition is more suitable for forecasting when the time series contains identifiable trend and seasonal components because these components can be analyzed separately.

Overall Conclusion

The Line Plot is best for observing the actual daily stock-price movement. The Moving Average Plot is useful for reducing short-term noise and identifying the underlying trend. STL Decomposition is the most comprehensive method because it separates trend, seasonality, and irregular variation.

Code

```
# ========================================
# FINANCIAL TIME SERIES ANALYSIS
# ALL 3 GRAPHS ON ONE PAGE
# ========================================

# Import dataset
```

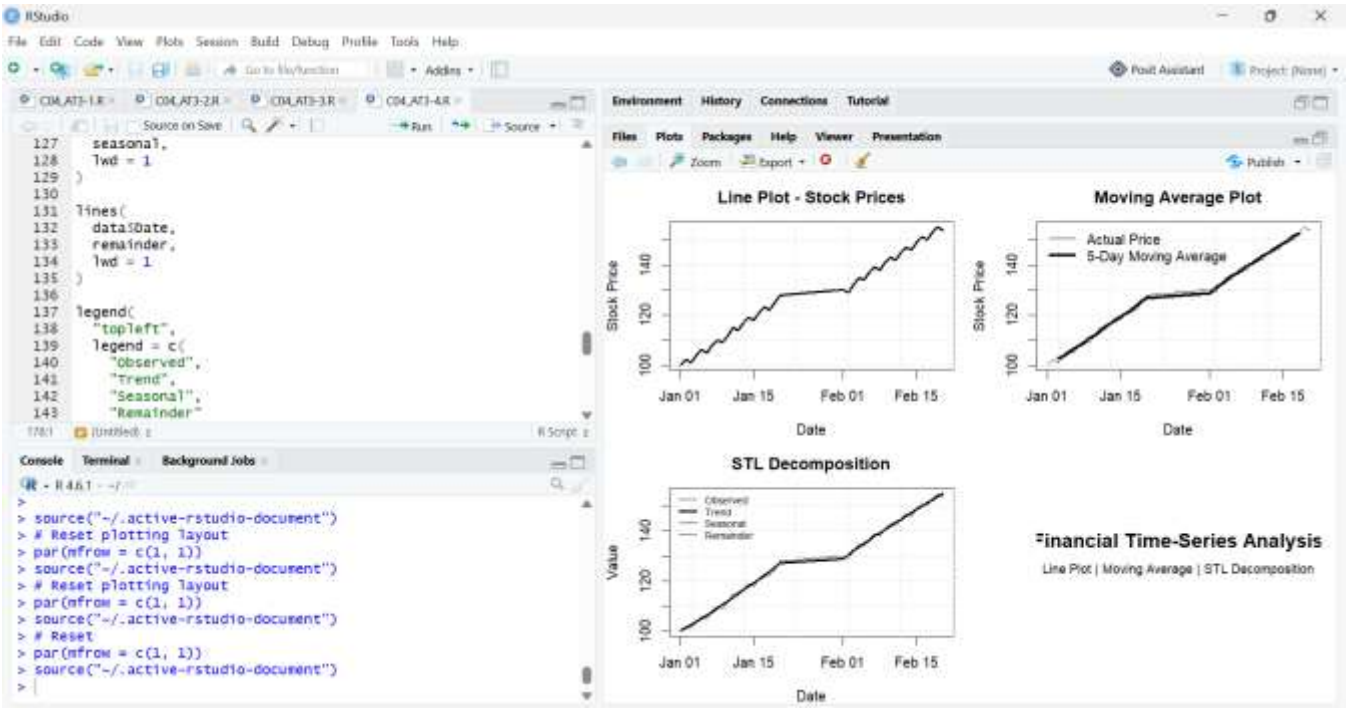
```
data <- read.csv(file.choose())

# Convert Date
data$Date <- as.Date(data$Date)

# Sort by date
data <- data[order(data$Date), ]

# Stock price
price <- data$Stock_Price


# ============================================
# 1. MOVING AVERAGE
# ============================================

moving_average <- stats::filter(
  price,
  rep(1/5, 5),
  sides = 2
)


# ============================================
# 2. STL DECOMPOSITION
# ============================================

stock_ts <- ts(price, frequency = 7)

stl_result <- stl(
  stock_ts,
  s.window = "periodic"
)

# Extract STL components
trend <- stl_result$time.series[, "trend"]
seasonal <- stl_result$time.series[, "seasonal"]
remainder <- stl_result$time.series[, "remainder"]


# ============================================
# 3. PUT ALL 3 GRAPHS ON ONE PAGE
# ============================================

par(
  mfrow = c(2, 2),
```

```
  mar = c(4, 4, 3, 1)
)


# ============================================
# GRAPH 1: LINE PLOT
# ============================================

plot(
  data$Date,
  price,
  type = "l",
  lwd = 2,
  main = "Line Plot - Stock Prices",
  xlab = "Date",
  ylab = "Stock Price"
)

grid()


# ============================================
# GRAPH 2: MOVING AVERAGE
# ============================================

plot(
  data$Date,
  price,
  type = "l",
  lwd = 1,
  main = "Moving Average Plot",
  xlab = "Date",
  ylab = "Stock Price"
)

lines(
  data$Date,
  moving_average,
  lwd = 3
)

grid()

legend(
  "topleft",
  legend = c("Actual Price", "5-Day Moving Average"),
```

```
  lty = 1,
  lwd = c(1, 3),
  bty = "n"
)


# ==========================================
# GRAPH 3: STL DECOMPOSITION
# ==========================================

plot(
  data$Date,
  price,
  type = "l",
  lwd = 1,
  main = "STL Decomposition",
  xlab = "Date",
  ylab = "Value"
)

lines(
  data$Date,
  trend,
  lwd = 2
)

lines(
  data$Date,
  seasonal,
  lwd = 1
)

lines(
  data$Date,
  remainder,
  lwd = 1
)

legend(
  "topleft",
  legend = c(
    "Observed",
    "Trend",
    "Seasonal",
    "Remainder"
  ),
```

```
  lty = 1,
  lwd = c(1, 2, 1, 1),
  bty = "n",
  cex = 0.7
)

grid()


# ============================================
# FOURTH PANEL
# ============================================

plot.new()

text(
  0.5,
  0.65,
  "Financial Time-Series Analysis",
  cex = 1.4,
  font = 2
)

text(
  0.5,
  0.45,
  "Line Plot | Moving Average | STL Decomposition",
  cex = 0.9
)


# Reset
par(mfrow = c(1, 1))
```

Output



5. Social Network Analysis

Comparative Analysis

Parameter	Network Graph	Force-Directed Graph	Adjacency Matrix
A. Relationship representation	Excellent	Excellent	Good
B. Community detection	Good	Excellent	Moderate
C. Scalability	Poor for very large networks	Moderate	Excellent
D. Visual clutter	Can become cluttered	Reduces clutter	Very low
E. Influential nodes	Good	Excellent	Good

A. Relationship Representation

The network graph directly displays users as nodes and interactions as edges. The force-directed graph improves this by positioning strongly connected users closer together. The adjacency matrix represents each relationship numerically as 1 or 0.

B. Community Detection

The force-directed graph is particularly useful for detecting communities because connected users naturally tend to form groups.

C. Scalability

For small networks, network graphs are easy to understand. As the number of users increases, the graph can become crowded. An adjacency matrix scales better because relationships are represented in a structured table.

D. Visual Clutter

A traditional network graph can become highly cluttered when there are many users and connections. The force-directed layout helps reduce overlapping edges, while the adjacency matrix avoids edge-crossing problems completely.

E. Influential Nodes

A highly connected node is potentially influential. In the code, node size in the force-directed graph is based on degree, so users with more connections appear larger.

Overall Conclusion

Network Graph → Best for direct relationship visualization
Force-Directed Graph → Best for communities and influential users
Adjacency Matrix → Best for large networks and precise relationship checking
For exploratory social-network analysis, the force-directed graph is generally the most informative, while the adjacency matrix is more scalable for large networks.

Code

```
# ==========================================
# SOCIAL NETWORK ANALYSIS
# Network Graph + Force-Directed Graph
# + Adjacency Matrix
# ALL 3 ON ONE PAGE
# ==========================================

# Import dataset
data <- read.csv(file.choose())

# Create list of users
users <- sort(unique(c(data$User1, data$User2)))

# Number of users
n <- length(users)


# ==========================================
# CREATE ADJACENCY MATRIX
```

```
# ==========================================

adj_matrix <- matrix(
  0,
  nrow = n,
  ncol = n,
  dimnames = list(users, users)
)

# Add connections
for (i in 1:nrow(data)) {

  u1 <- data$User1[i]
  u2 <- data$User2[i]

  adj_matrix[u1, u2] <- 1
  adj_matrix[u2, u1] <- 1
}


# ==========================================
# SET ONE PAGE
# ==========================================

par(
  mfrow = c(2, 2),
  mar = c(4, 4, 4, 2)
)


# ==========================================
# 1. NETWORK GRAPH
# ==========================================

set.seed(123)

x <- runif(n)
y <- runif(n)

plot(
  x,
  y,
  type = "n",
  xlab = "",
  ylab = "",
  xaxt = "n",
```

```
  yaxt = "n",
  main = "Network Graph"
)

# Draw connections
for (i in 1:n) {
  for (j in i:n) {
    if (adj_matrix[i, j] == 1) {
      segments(
        x[i], y[i],
        x[j], y[j],
        col = "gray"
      )
    }
  }
}

# Draw nodes
points(
  x,
  y,
  pch = 19,
  cex = 2,
  col = "steelblue"
)

# Add labels
text(
  x,
  y,
  labels = users,
  pos = 3,
  cex = 1
)


# ============================================
# 2. FORCE-DIRECTED GRAPH
# ============================================

# Start with random positions
fx <- runif(n)
fy <- runif(n)

# Simple force simulation
for (iteration in 1:100) {
```

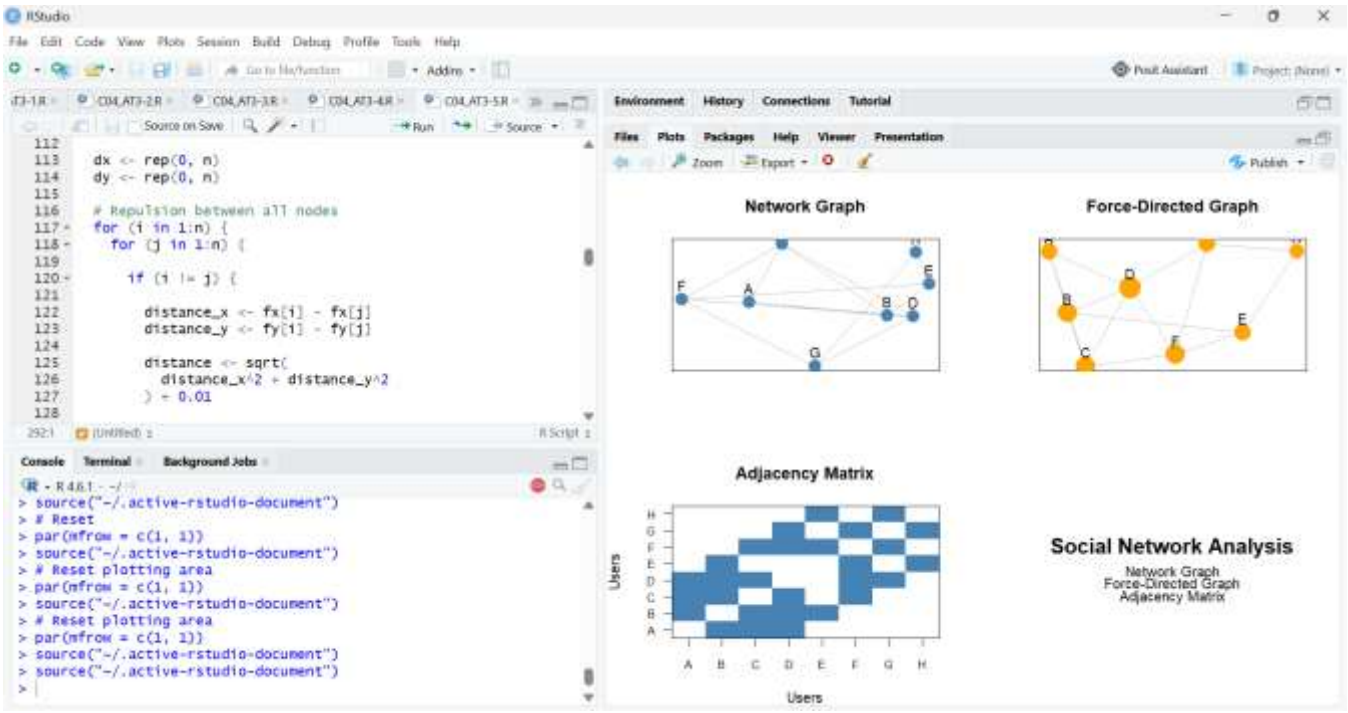
```
dx <- rep(0, n)
dy <- rep(0, n)

# Repulsion between all nodes
for (i in 1:n) {
  for (j in 1:n) {

    if (i != j) {

      distance_x <- fx[i] - fx[j]
      distance_y <- fy[i] - fy[j]

      distance <- sqrt(
        distance_x^2 + distance_y^2
      ) + 0.01

      force <- 0.002 / distance^2

      dx[i] <- dx[i] +
        (distance_x / distance) * force

      dy[i] <- dy[i] +
        (distance_y / distance) * force
    }
  }
}

# Attraction between connected nodes
for (i in 1:n) {
  for (j in 1:n) {

    if (adj_matrix[i, j] == 1) {

      distance_x <- fx[j] - fx[i]
      distance_y <- fy[j] - fy[i]

      fx[i] <- fx[i] +
        distance_x * 0.01

      fy[i] <- fy[i] +
        distance_y * 0.01
    }
  }
}
```

```
  fx <- fx + dx
  fy <- fy + dy
}

# Normalize positions
fx <- (fx - min(fx)) /
  (max(fx) - min(fx))

fy <- (fy - min(fy)) /
  (max(fy) - min(fy))

# Plot
plot(
  fx,
  fy,
  type = "n",
  xlab = "",
  ylab = "",
  xaxt = "n",
  yaxt = "n",
  main = "Force-Directed Graph"
)

# Draw edges
for (i in 1:n) {
  for (j in i:n) {

    if (adj_matrix[i, j] == 1) {

      segments(
        fx[i], fy[i],
        fx[j], fy[j],
        col = "gray"
      )
    }
  }
}

# Calculate degree
degree <- rowSums(adj_matrix)

# Node size based on connections
node_size <- 1.5 + degree * 0.4

# Draw nodes
points(
```

```
  fx,
  fy,
  pch = 19,
  cex = node_size,
  col = "orange"
)

# Labels
text(
  fx,
  fy,
  labels = users,
  pos = 3,
  cex = 1
)


# ============================================
# 3. ADJACENCY MATRIX VISUALIZATION
# ============================================

image(
  1:n,
  1:n,
  t(adj_matrix),
  col = c("white", "steelblue"),
  axes = FALSE,
  main = "Adjacency Matrix",
  xlab = "Users",
  ylab = "Users"
)

axis(
  1,
  at = 1:n,
  labels = users,
  cex.axis = 0.8
)

axis(
  2,
  at = 1:n,
  labels = users,
  las = 1,
  cex.axis = 0.8
)
```

```
box()


# ============================================
# 4. INFORMATION PANEL
# ============================================

plot.new()

text(
  0.5,
  0.70,
  "Social Network Analysis",
  cex = 1.5,
  font = 2
)

text(
  0.5,
  0.50,
  "Network Graph",
  cex = 1
)

text(
  0.5,
  0.40,
  "Force-Directed Graph",
  cex = 1
)

text(
  0.5,
  0.30,
  "Adjacency Matrix",
  cex = 1
)


# Reset plotting area
par(mfrow = c(1, 1))
```

Output


RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
112
113 dx <- rep(0, n)
114 dy <- rep(0, n)
115
116 # Repulsion between all nodes
117 for (i in 1:n) {
118 for (j in 1:n) {
119
120 if (i != j) {
121
122 distance_x <- fx[i] - fx[j]
123 distance_y <- fy[i] - fy[j]
124
125 distance <- sqrt(
126 distance_x^2 + distance_y^2
127) + 0.01
128
Console Terminal Background Jobs
> source("~/.active-rstudio-document")
> # Reset
> par(mfrow = c(1, 1))
> source("~/.active-rstudio-document")
> # Reset plotting area
> par(mfrow = c(1, 1))
> source("~/.active-rstudio-document")
> # Reset plotting area
> par(mfrow = c(1, 1))
> source("~/.active-rstudio-document")
> source("~/.active-rstudio-document")
Environment History Connections Tutorial
Files Plots Packages Help Viewer Presentation
Zoom Export Publish
Network Graph
Force-Directed Graph
Adjacency Matrix
Users
Social Network Analysis
Network Graph
Force-Directed Graph
Adjacency Matrix

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks	
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant		
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison		
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided		
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation		
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand		

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____